FPGA Implementation of a Modular Active Noise Control System

Shiva Gholami Boroujeny*, Mohammad Eshghi**
Electrical and Computer Engineering Faculty, Shahid Beheshti University, Evin, Tehran, Iran
*sh_gholami@sbu.ac.ir
**m-eshghi@sbu.ac.ir

Abstract: *This contribution presents the High-speed field programmable gate array (FPGA) implementation of a modular architecture of the Active Noise Control (ANC) system with online secondary path modelling.The clock frequency that is obtained for this FPGA implementation is 120MHz. A comparison of this implementation to other FPGA and DSP implementations of an ANC system shows a better speed up and convergence time for the proposed design. This innovative modular implementation of the ANC system results in a fast design and fast convergence with capability to expand the four adaptive filters and data bus when higher speed for ANC system is desired. In the other word, considering the modularity of the design and the reconfigurability of FPGA, one can expand an ANC system for different accuracies and required convergence times.*

Keywords: active noise control, modular ANC system, FPGA implementation, adaptive filter.

1. Introduction

The reduction of Acoustic noise has become a serious problem as the number of industrial equipment such as engines, blowers, fans, compressors and transformers are increased. In recent years powerful digital signal processing (DSP) devices have made possible the development of real time ANC systems with a wide range of applications. DSP algorithms have been developed for single and multiple channel ANC systems in branches of broadband, narrowband and adaptive feed-forward control [1-4].

Authors in [5] presented a hardware implementation of an ANC system using TMS320C25 DSP processor. However, with its growing die size as well as incorporating the embedded digital signal processing blocks, the FPGA devices have become a serious contender in the signal processing market. The processor in a DSP approach needs some time to execute the software program. The FPGA approach, in the other hand, performs the tasks on a hardware base and it is faster than the DSP approach. Because of the need for much speed in these systems, FPGA devices are good candidates for these applications.

The authors in [6] implemented a one channel ANC system on Xilinx Spartan2E-300 FPGA. Recently, two design and FPGA implementation for ANC systems are proposed [7,8]. To achieve the better performance of ANC systems, some modifications are made to these systems [9-11]. Researchers in [9] proposed a new method using online secondary path modeling to reduce the residual noise in an active noise control system.
In all of these designs, the architecture of ANC system is not modular. A modular architecture for an ANC system using secondary path modeling is presented in [12]. This design comprises of four similar modules to reduce the unwanted primary noise. One of these modules performs as the main adaptive control filter that uses an Fx-LMS algorithm to converge the tap-weights. Two modules work as the adaptive modeling filters that use an LMS algorithm to model the secondary path filter. The last module works as a digital filter to produce the anti-noise output signal. Besides of low complexity and simplicity of implementation, this configurable design provides a mean to implement a fast ANC system on ASIC or FPGA technologies.

This paper proposes the FPGA implementation of the modular architecture [12] of the ANC system. This new modular implementation of the ANC system results in a fast design and fast convergence with capability to expand the four adaptive filters and data bus when higher speed for ANC system is desired.

This paper is organized as follows. In section 2, we present a summary of the modular architecture for an ANC system. Implementation results of the modular ANC system are considered in section 3. Comparison of the results is presented in section 4 and the conclusion is in section 5.

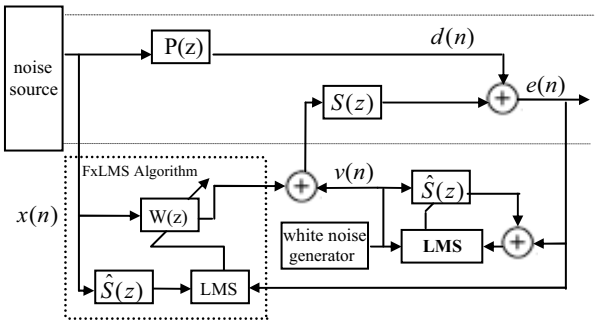

Fig. 1: ANC system with online secondary path modeling

Proceedings of ICEE 2010, May 11-13, 2010
978-1-4244-6760-0/10/$26.00 ©2010 IEEE

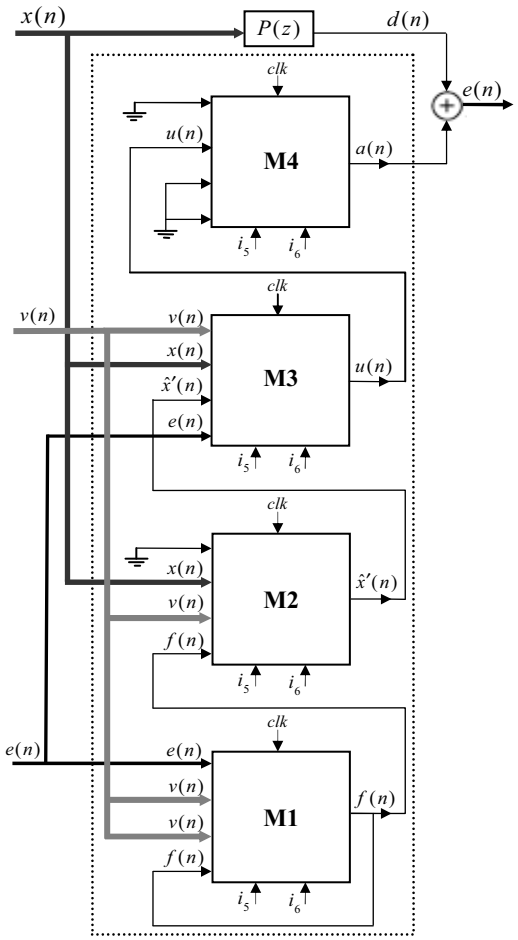

Fig. 2: Architecture of the ANC system with online secondary path modeling using four M modules

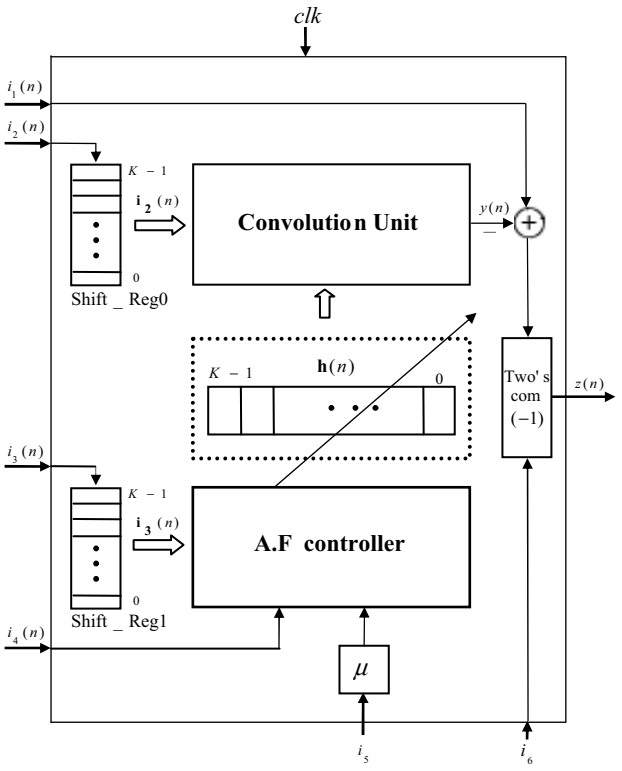

Fig. 3: Block diagram of the module M

2. A Summary of Modular Design of an ANC System

An online secondary-path modeling technique using additive random noise [1, 2] is shown in Figure 1.

This ANC system has three inputs and one output. The inputs of this ANC system are the reference noise signal, ***x(n)***, the injected noise signal, ***v(n),*** and the error microphone output, *e(n)*, inputted to the ANC system. The output of this system is the anti-noise signal, which is propagated by the loudspeaker and combines with the primary noise signal to reduce the noise pressure around the error microphone.

The modular design [12] for this ANC system consists of four similar modules. The architecture of these modules is the same, but their inputs and outputs are different. These four modules are composed to form an ANC system with online secondary path modeling. The detail of this ANC system using the four M modules is shown in Figure 2.

In this Figure, *P(z)* is the primary acoustic path between the reference noise source and the error microphone. The reference noise signal, ***x(n)***, is filtered through *P(z)* and appears as a primary noise signal at the error microphone. The output of the primary path filter, *d(n)*, is the desired output of the ANC system which is the convolution of the reference signal, ***x(n)*** and the tap-weights of the primary path FIR filter, ***P(n)***, i.e.

$$d(n) = \sum_{k=0}^{N-1} p_k(n)x(n-k) \qquad (1)$$

where *N*, is the length of the primary path FIR filter.

The objective of the ANC system is to generate an appropriate anti-noise signal, *a(n)*, propagated by the loudspeaker to create a zone of silence in the vicinity of the error microphone. The module M used in this design has six inputs and one output. The block diagram of this module is shown in Figure 3.

Each module consists of two shift registers, a convolution unit, an adaptive filter controller, a 2's complementer, and an adder.

The first input to the module, $i_1(n)$, represents the desired output of the adaptive filter at time *n*. The second input, $i_2(n)$, represents the input data to the adaptive filter at time *n*. The third input, $i_3(n)$, represents the modified input data to the adaptive filter at time *n*. The forth input, $i_4(n)$, represents the error data at time *n*. The fifth input, i_5, is the input data to the step size parameter register. The sixth input, i_6, is a control signal to negate the output of the module.

The output of the module, *z(n)*, is the difference between the adaptive filter's output and the desired output at time *n*.

Two shift registers with the length of *K* are installed in the inputs of the block to save and shift the input data and the modified input data at each time *n*.

TABLE I: Inputs, output and computations of four modules

	$i_1(n)$	$i_2(n)$	$i_3(n)$	$i_4(n)$	i_5	i_6	$z(n)$	Con. unit	Adaptation
M1	$e(n)$	$v(n)$	0	0	0	0	$f(n)=e(n)-\hat{v}'(n)$	$\hat{v}'(n)=\sum_{m=0}^{M-1}\hat{s}_m(n)v(n-m)$	—
M2	0	$x(n)$	$v(n)$	$f(n)$	0	1	$\hat{x}'(n)$	$\hat{x}'(n)=\sum_{m=0}^{M-1}\hat{s}_m(n)x(n-m)$	$\hat{\mathbf{s}}(n+1)=\hat{\mathbf{s}}(n)+\mu_s f(n)\mathbf{v}(n)$
M3	$v(n)$	$x(n)$	$\hat{x}'(n)$	$e(n)$	1	0	$u(n)=v(n)-y(n)$	$y(n)=\sum_{l=0}^{L-1}w_l(n)x(n-l)$	$\mathbf{w}(n+1)=\mathbf{w}(n)+\mu_w e(n)\hat{\mathbf{x}}'(n)$
M4	0	$u(n)$	0	0	0	0	$a(n)=0-u'(n)$	$u'(n)=\sum_{l=0}^{L-1}s_l(n)u(n-l)$	—

The digital filter *H(z)* contains *K* tap-weights. The output of this unit at time *n* is the convolution of the input vector, $\boldsymbol{i}_2(n)$ and the tap-weight vector of the filter, $\boldsymbol{h}(n)$, i.e.

$$y(n)=\sum_{j=0}^{K-1}h_j(n)i_2(n-j) \tag{2}$$

The tap-weights of this digital filter, $\boldsymbol{h}_i(n)$, are updated at each time using the Least Mean Square (LMS) algorithm, i.e.

$$h_k(n+1)=h_k(n)+\mu i_3(n)i_4(n-k),\quad k=0,...,K\text{-}1 \tag{3}$$

Where μ is the step size in the LMS algorithm, and $i_3(n)$ and $i_4(n)$ are the input values to the module at time *n*.
If the control input to 2's complementer unit, i_6, is one, then the 2's complement of the data input is obtained in the output. The adder in the module subtract the value of the output of the adaptive filter and the input $i_1(n)$, i.e.

$$z(n)=i_1(n)-y(n) \tag{4}$$

The output of the adder is the output of the module. Four of these modules are used to form an ANC system with online secondary path modeling as shown in Figure 2.
The inputs, output and computations of these four modules are depicted in Table I.

TABLE II: The number of elements used on Stratix FPGA, $\mu_w=2^{-11}$, $\mu_{\hat{s}}=2^{-6}$

Resource	Used	Avail	Utilization
IOs	103	1,171	8 %
Registers	4,050	150,386	2 %
ALUT	6,344	143,520	4%
DSP block 9-bit elems	736	768	95%

3. Implementation

In this section the modular ANC system with online secondary path modeling is implemented on FPGA.

The FPGA device that is chosen in this implementation is a Stratix family EP2S180F1508C4 FPGA. These FPGAs have embedded DSP blocks which have dedicated multiplier, pipeline and accumulator circuitries. With the embedded DSP blocks, these FPGAs can perform high speed calculations, therefore are useful devices for digital signal processing designs, as well as adaptive filters [13].

The step size parameters of the adaptive control filter, μ_w, and the adaptive modeling filter, $\mu_{\hat{s}}$, are considered to be 2^{-11} and 2^{-6}, respectively. All the signals in ANC processor are considered to be floating point numbers with 32 bit length that comprises 11 accuracy bits. The number of used DSP blocks for implementing the ANC system on FPGA is 736 blocks. The clock frequency is 120MHz. A summary of implementation results is shown in Table II.

For confidence of true performance of the implemented ANC system, a comparison between MATLAB simulation and VHDL implementation is done. The corresponding curves of residual error signals, *e(n)*, are shown in Figure 4.

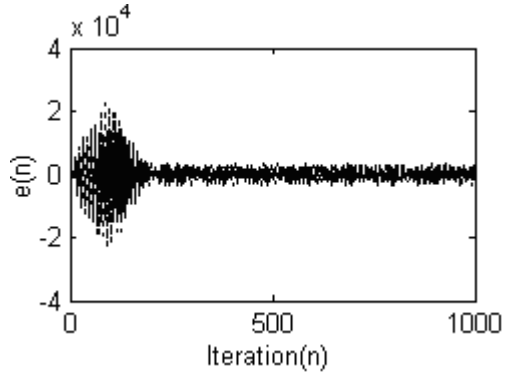

Fig. 4: Comparison of MATLAB simulation and VHDL implementation
continues line: MATLAB, dashed line: VHDL

4. Comparison

To evaluate the proposed implementation, the results were compared with the results of implementations presented in [5] and [6]. The ANC systems in these researches have no secondary path, where, our ANC system has secondary path modelling filter.

A hardware implementation of an ANC system using TMS320C25 DSP processor was reported in [5]. The

TMS320C25 family of digital signal processors uses a programmable, 16-bit, fixed-point format [14] with the clock frequency of 40MHz. The computational time needed to execute an iteration of the ANC system on this DSP processor is about 78.1 usec. The authors showed that for an acceptable convergence, 350 iterations is needed, leads to a total convergence time of about 27.3 ms (=350*78.1 us).

The authors in [6] implemented a one-channel ANC system on Xilinx Spartan2E-300 FPGA. The order of the adaptive filter for this system is chosen to be M=32. The results of the FPGA implementation reported in [6] shows that each iteration needs 4 clock pulses. Since the clock frequency of the system is 50MHz, the time for each iteration is 80 nsec. The authors showed that for an acceptable convergence, 1200 iterations is needed, leads to a total convergence time of about 96 us (= 1200*80 ns).

The proposed FPGA implementation of the modular ANC system in this paper uses a 32-bit fixed point format. The adaptive control filter and the adaptive modeling filter have 32 tap-weights. The clock frequency that obtained for this FPGA implementation is 120 MHz. The system needs 66.7 ns to execute an iteration. The proposed design needs 190 iterations for an acceptable convergence. This results lead to a total convergence time of about 12.67us (= 190*66.7ns).

The above calculation shows that the proposed design and its implementation on FPGA have a speedup of 2154 (=27.3ms/12.67us) with respect to the DSP implementation [5] and a speed up of 7.5 (96us/12.67us) with respect to implementation reported in [6].

5. Conclusion

In this paper a hardware implementation of the modular ANC system with online secondary path modeling was presented. This ANC system contains four similar modules to reduce the unwanted primary noise. Each module consists of two shift registers, a convolution unit, an adaptive filter controller, an adder, and a two's complementer. One of these modules per forms as the main adaptive control filter that uses an FxLMS algorithm to converge the tap-weights. Two modules work as the adaptive modeling filters, used LMS algorithm to model the secondary path filter. The last module works as a digital filter to produce the anti-noise output signal.

The ANC system was implemented on the Altera FPGA. The current FPGA implementation of the ANC system uses a 32- bit fixed point format. The adaptive control filter and the adaptive modeling filter have 32 tap-weights. The clock frequency that obtained for this FPGA implementation is 120 MHz. The system needs 66.7 ns to execute an iteration. The proposed design needs 190 iterations for an acceptable convergence. This results lead to a total convergence time of about 12.67us (=190*66.7ns). The implementation results were compared with the simulation results using MATLAB.

To evaluate the proposed design, the results of this implementation were compared with the results of implementation designs presented in [5] and [6]. The total convergence time of an ANC system, reported in [5] and [6] are 27.3ms and 96us, respectively. As a result, the proposed design and its implementation achieved speed up of about 2154 with respect to the implementation reported in [5], and a speed up of 7.5 with respect to the implementation reported in [6].

Acknowledgements

This research is supported by Iran Telecommunication Research Center (ITRC).

References

[1] S. M. Kuo, D.R. Morgan, *Active Noise Control Systems-Algorithms and DSP Implementation*, New York: Wiley, 1996.

[2] S. M. Kuo, and D. R. Morgan, "Active Noise Control: A tutorial review," Proc. *IEEE*, vol. 87, PP. 943-973, 1999.

[3] S. M. Kuo, and D. R. Morgan," Review of DSP Algorithms for Active Noise Control," *IEEE Control Application*, 2000.

[4] F. Nagi, J. Low, S. Li , "Active Noise Cancellation with TMS320 C5402 DSP Starter Kit," *IEEE Signal processing* , 2004.

[5] S. M. Kuo, I. Panahi, K. Chung, T. Horner, J. Chyan, "Design of Active Noise Control Systems With the TMS320 Family," *Texas Instruments, Digital Signal Processing Products*, SPRA042, June 1996.

[6] A. D. Stefano, A. Scaglione, C. Giaconia, "Efficient FPGA Implementation of an Adaptive Noise Canceller," in *Proc. IEEE 2005, CAMP'05*, vol. 00, 2005, pp. 87-89.

[7] A. Jalali, Sh. Gholami Borouj and M. Eshghi, "Design and Implementation of a Fast Active Noise Control System on FPGA," in *Proc. IEEE MED'07*, Athens, Greece, 2007.

[8] M. Eshghi, Sh. Gholami Boroujeny and A. Jalali, "A Hardware Design for an Online Active Noise Control System," *IEEE MED'07 Conference*, Athens, Greece, 2007.

[9] M. T. Akhtar, M. Abe, and M. Kawamata, "A New Structure for Feedforward Active Noise Control Systems With Improved Online Secondary Path Modeling," IEEE Trans. Speech Audio Process., vol. 13, 2003, pp. 155-158 .

[10] M. Zhang, H. Lan, and W. Ser, "Cross-updated active noise control system with online secondary path modeling," *IEEE Trans. Speech Audio Process*., vol. 9, pp. 598–602, July 2001.

[11] A. Carini, S. Malatini and G. Sicuranza, "Optimal Variable Step-Size NLMS Algorithms for Feedforward Active Noise Control," In *Proc EUSIPCO 2008*, Lausanne Switzerland, August 25-29. 2008.

[12] Sh. Gholami Boroujeny, M. Eshghi, "A Modular Design for an Active Noise Control System," *IEEE TENCON Conference*, Hyderabad, India, November 2008.

[13] Altera, Stratix Programmable Logic Device Family Data Sheet, Data Sheet DS-STXFAMLY-2.1, Altera, Inc., August, 2002.

[14] Texas instrument, Incorporated, TMS320 Second-Generation Digital Signal Processors, November 1990.